Trimming the Block-Scale Tax in a SpaceMiT X60 IME `int8×int4` GEMM Kernel: a correct, bit-exact optimization with no end-to-end uplift

opensolvers/benchmarks — IME kernel study

July 2026

Abstract

llama.cpp ships a hand-written matrix-extension (IME) backend for the SpaceMiT X60 (Ky X60) RISC-V core, built on the vendor `smt.vmadot` instruction. Its block-scaled Q4_0 GEMM kernel (`gemm_kernel_i8i4`) runs at roughly 0.6× the throughput of an equivalent pure-integer `s8s8s32` kernel using the same `smt.vmadot` peak. We attribute that gap on real silicon: stripping the per-block floating-point reduction shows the tax is ~31–37 %, and it is *dominated by the per-block scale build*, not the integer→float convert and accumulate. Because throughput falls off almost linearly with removed instructions, the X60 evidently does **not** overlap the IME pipe with the RVV floating-point pipe, so cutting scale-build instruction count directly buys time. We rewrite the scale build to use eight `vfmul.vv` against two prebuilt row-scale vectors instead of sixteen masked `vfmul.vf` plus four vector moves, preserving the exact accumulator layout. The result is **bit-identical** to the stock kernel (verified by a signature A/B on the board) and ~**5 % faster** on both `M4` prefill paths. Yet an end-to-end `llama-bench` evaluation of Qwen2.5-0.5B on the X60 shows **no measurable prompt-processing uplift**: the prefill throughput is statistically tied across thirteen interleaved A/B rounds, because per-run contention noise (±15–20 %) dwarfs the ~3–4 % an isolated GEMM speedup can contribute to a prefill that is only partly matmul. This is, deliberately, a *negative result at the application level* accompanied by a *positive, reproducible one at the kernel level* — and an argument for reporting both.

1. Introduction

The SpaceMiT X60 ("Ky X60", the core in the K1/M1 SoC and the Banana Pi / Orange Pi RV2 boards) is an eight-core RISC-V application processor implementing the RVV 1.0 vector extension at `VLEN=256`, plus a vendor **Integer Matrix Extension (IME)** exposed through the custom instruction `smt.vmadot`.

A single `smt.vmadot` performs a 4×4 `int32` tile update from two 4×8 `int8` operands, and is the throughput engine behind the "2 TOPS AI" figure quoted for the part.

llama.cpp is, at the time of writing, the only mainstream inference stack that ships an X60 IME backend (`ggml-cpu/spacemit`, guarded by `GGML_CPU_RISCV64_SPACEMIT`). Its central kernel, `spacemit_kernels::ime1::gemm_kernel_i8i4`, is a *block-scaled* Q4_0 matrix multiply: `int8` activations times 4-bit weights, with a per-32-element `fp16` scale, accumulated in `fp32`. A companion microbenchmark in this repository (`ime/`) established that this block-scaled kernel tops out around 0.6× the throughput of a pure `s8s8s32` kernel built on the same `smt.vmadot` peak. The difference is the *block-scale tax*: work the integer kernel never does — one `int32`→`fp32` convert and one scale-multiply per 32-element K-block, per output tile.

Prior notes speculated that the tax "needs a kernel rewrite, not a driver tweak." This paper does the rewrite, measures it rigorously on the board, and reports the outcome honestly. The kernel-level result is a clean, bit-exact ~5 %; the application-level result is *nothing measurable*. We consider the second finding as important as the first: it is a concrete, root-caused example of a microbenchmark win that does not survive contact with a real workload on real, shared hardware, and it explains precisely why.

2. Background

2.1 The IME kernel path

`gemm_kernel_i8i4` dispatches on the number of activation rows:

- `M4` (`count_m` ≥ 4) — the register-blocked path used by *prefill* (`pp`, prompt processing). It computes a 4×16 `fp32` output tile per call and dominates prompt-processing compute. It has two variants, with and without a per-block zero point (`M4` no-ZP is the one Q4_0 uses; `M4` ZP serves zero-point quant formats).
- `M1` (`count_m < 4`) — the GEMV path used by *decode* (`tg`, token generation), one activation row against the weight matrix.

Per K-block, the `M4` path issues sixteen `smt.vmadot` (the matmul), then a floating-point *reduction* consisting of (i) a **scale build** (`LOAD_SCALE_4x16_FP16`) that loads sixteen `fp16` weight scales, widens them, and folds in the four per-row activation scales, and (ii) a **convert + accumulate** (one `vfcvt.f.x.v` and one `vfmacc.vv`, both at `LMUL=8`) that turns the `int32` tile accumulator into scaled `fp32` and adds it into the running output. The integer accumulator must reset every K-block because each Q4_0 block carries its own scale — this is the irreducible structural reason the reduction cannot be hoisted across blocks.

2.2 The accumulator layout (why the scale build is awkward)

`smt.vmadot` writes a 4×4 tile into an even/odd register pair. The four accumulators `v16/v18/v20/v22` therefore hold four *column-groups*, each 4 rows $\times$ 4 cols, and the store macro `SAVE_RESULT_4x16` reveals the exact lane mapping: within each `fp32` output register, the low four lanes are one output row and the high four lanes are the next row, for a fixed column-group. The combined scale for that register is thus $A_{\text{scale}}[\text{row}] \cdot B_{\text{scale}}[\text{col}]$ with the *weight* scale replicated across the two row-halves and the two *activation* scales applied per-half. The stock code realises this with overlapping masked loads (to replicate the weight scales) followed by sixteen predicated `vfmul.vf` (eight at `vl=4`, eight at `vl=8` under a `0xf0` mask) to apply the row scales — plus four `vmv.v.v` copies and several `vsetvli` (vector-type) switches.

3. The block-scale tax, measured

All measurements are single-core on an Orange Pi RV2 (X60 @ 1.6 GHz, `performance` governor), pinned to core 0 of cluster 0, reported as the *clean peak* (the maximum over ≥ 4 process launches; contention only ever lowers a run, so the maximum is the true single-core capability). Throughput is single-core GOP/s ($2MNK/t$).

To localize the tax we build variants of the `M4` no-ZP path that strip parts of the per-block reduction (timing-only; these variants are numerically wrong by construction):

- **`-scale-build`**: remove `LOAD_SCALE_4x16_FP16`, keep the convert + accumulate.
- **`-all FP` (vmadot only)**: remove the entire reduction, leaving only the sixteen `smt.vmadot` and the loop overhead.

$M \times N \times K$	stock	−scale-build	−all FP (vmadot only)
512^3	22.5	29.6	32.7
768^3	27.5	36.7	41.3
$512 \times 512 \times 2048$	27.5	38.7	43.6

Two things follow. First, the full floating-point tax is large — 31–37 % — and the `vmadot`-only ceiling at 768^3 (41.3) and large K (43.6) approaches the pure `s8s8s32` ceiling (~42), confirming the matmul engine itself is not the bottleneck. Second, and decisively: removing ~40 scale-build instructions recovers ~31 %, whereas removing the ~16-instruction convert + accumulate recovers only ~10 %. The recovered time is roughly proportional to removed instruction count. **The**

X60 does not appreciably overlap the IME pipe with RVV floating-point — the two phases are effectively serial — so the tax is paid in issue slots, and cutting instruction count in the dominant phase (the scale build) is the lever.

4. Optimization: a register-efficient scale build

The scale build produces eight `fp32` registers `v8..v15`, each holding B_{scale}[col-group] replicated across both row-halves and scaled by two activation scales. The stock code applies the row scales with sixteen predicated `vfmul.vf` (half of them at `vl=8` under a mask, i.e. doing four lanes of work in eight lanes of time) plus four `vmv.v.v` and repeated `vsetvli` switches.

We instead precompute two *row-scale vectors* once per K-block — $\mathbf{a}_{\text{even}} = [A_0^{\times 4}, A_1^{\times 4}]$ and $\mathbf{a}_{\text{odd}} = [A_2^{\times 4}, A_3^{\times 4}]$ — with two `vfmv.v.f` and two `vfmerge.vfm`, then form the eight combined-scale registers with eight full `vfmul.vv`:

```
vfmv.v.f     v2, f1            ; a_even = [A0,A0,A0,A0, A1,A1,A1,A1]
vfmerge.vfm  v2, v2, f2, v0    ;   (v0 = 0xf0 mask -> high half)
vfmv.v.f     v3, f3            ; a_odd  = [A2,A2,A2,A2, A3,A3,A3,A3]
vfmerge.vfm  v3, v3, f4, v0
vfmul.vv     v8,  v4, v2       ; B(cg0) * a_even
vfmul.vv     v9,  v4, v3       ; B(cg0) * a_odd
vfmul.vv     v10, v5, v2       ; ... four column-groups x {even,odd}
...
vfmul.vv     v15, v7, v3
```

This drops the four `vmv.v.v`, halves the multiply count (16 masked `vfmul.vf` → 8 full `vfmul.vv`), and removes the `vl=4`/`vl=8` `vsetvli` alternation. The `v8..v15` layout is byte-for-byte the layout the stock kernel produced, so the downstream `vfcvt`/`vfmacc` and `SAVE_RESULT_4x16` are unchanged and the numeric result is identical. The change is ~40 lines confined to a single macro (patch: `ime/llama-ime1-scalebuild-opt.patch`).

Both `M4` variants (no-ZP and ZP) share `LOAD_SCALE_4x16_FP16`, so the same macro applies to both; the swap is register-safe in the ZP path (it preserves `v1`, the zero-point gather-index register, and clobbers only registers the stock macro already clobbers).

5. Methodology

Build. The kernel is edited on an x86-64 host and cross-compiled with the SpaceMiT GCC toolchain into `libggml-cpu.so`; an incremental rebuild is ~20 s. The shared object is copied to the board, where the microbenchmark and `llama-bench` load it via `rpath`.

Correctness. The microbenchmark harness (`ime/bench_i8i4.cpp`) gained

a `chk` mode that fills both operand buffers with varied bytes constrained to `[1,0x3f]` — guaranteeing every `fp16`/`fp32` value the kernel might read is finite and normal, while still varying per (row, column, block) so a wrong scale or tile layout surfaces as a mismatch. It then prints a signature (sum, sum-of-squares, max of the output). We A/B the *modified* kernel's signature against the *stock* kernel's on byte-identical inputs; a rewrite is accepted only if the signatures match exactly. Throughput is data-independent (straight-line integer and floating-point, no data branches), so timing is measured separately with representative data. The `chk` mode also gained `zp` and `m1` selectors so all three kernel paths can be exercised.

6. Kernel results

Signatures matched exactly for every size on both `M4` paths — the optimization is **bit-identical** to the stock kernel. Clean-peak throughput (paired A/B, single core):

kernel path	used by	512^3	768^3
`M4` no-zero-point	Q4_0 prefill	23.0 → 24.1 (+4.8 %)	27.6 → 29.2 (+5.8 %)
`M4` zero-point	ZP-quant prefill	20.6 → 21.7 (+5.3 %)	24.7 → 26.0 (+5.3 %)
`M1` / GEMV	decode (`tg`)	12.1 — N/A	—

The `M4` gain is a consistent ~5 % across 512^3–$2048^2 \times 512$. The `M1` / GEMV path is **not applicable**: its scale build is already minimal (four `vfmul.vf` for a single activation row, no masking, no `vmv`), so there is no sixteen-`vfmul` tax to cut, and GEMV on this part is memory-bound anyway (12.1 GOP/s, roughly half the `M4` rate — the weight stream, used once per token, dominates).

7. End-to-end evaluation

We ran `llama-bench` on Qwen2.5-0.5B (Q4_0) with the stock and patched `libggml-cpu.so`, four threads pinned to cluster 0 (`taskset -c 0-3`), comparing the two shared objects in the same session. `pp512` (prefill) exercises the optimized `M4` GEMM; `tg128` (decode) exercises the untouched `M1` path and serves as a negative control.

Correctness holds end-to-end. The patched build produces coherent, correct output (*"The capital of France is Paris."*) and `tg128` is bit-stable at **7.25 t/s** on both builds — as expected, since decode never touches the changed code.

Prefill shows no measurable uplift. Over thirteen interleaved A/B rounds:

metric	stock	patched
`pp512` mean (t/s)	79.8	79.5
`pp512` median (t/s)	82.3	79.9
`pp512` peak (t/s)	**91.3**	86.3
`pp512` range (t/s)	62.8–91.3	67.0–86.3
`tg128` (t/s)	7.25 ± 0.01	7.25 ± 0.01

The two prompt-processing distributions are statistically indistinguishable — means within 0.3 %, and if anything the stock peak and median are marginally higher (noise). The cause is visible in the range: `pp512` swings ±15–20 % run-to-run, a ~45 % total spread, because prefill is memory- and L2-bandwidth-heavy and this is a shared, multi-tenant part — the four cluster-0 cores share one 512 KB L2, and buffer placement additionally triggers the `malloc`-dependent cache-set aliasing documented in the companion `ime/` study. That noise floor is an order of magnitude larger than the uplift a 5 % *kernel* speedup can contribute to a prefill that is only partly matmul (attention, RoPE, softmax, norms, and the sampling path also cost). The expected end-to-end gain, ~3–4 %, is simply not resolvable here.

8. Discussion

Why the tax is otherwise fundamental. The convert and accumulate (`vfcvt`/`vfmacc`) and the weight-scale loads are near-irreducible, and the per-block scale cannot be deferred across K-blocks because each Q4_0 block has its own scale. The one lever that could hide the *whole* floating-point phase — software-pipelining the next block's `smt.vmadot` under the current block's reduction, given that the two pipes are serial — is **register-blocked**. A whole 4×16 tile needs eight accumulator registers; double-buffering it needs sixteen, but the `fp32` output (`v24..v31`), the `int32` accumulator (`v16..v23`), and the scale vectors (`v8..v15`) already fill the 32-register file.

Why offloading the reduction to the idle cores does not help. The four non-IME cores (cluster 1) sit idle during IME GEMM, which invites moving the floating-point reduction there. It does not pay: the two clusters have *separate* 512 KB L2 caches, and block scaling forbids accumulating `int32` across blocks, so an IME core would have to ship a per-block `int32` partial tile to a floating-point core — $M \cdot N \cdot K/8$ bytes in total (~16 MB at 512^3 versus a 1 MB output), across the inter-cluster interconnect, to save arithmetic that is only ~0.3 flop/byte. The intermediate lives in vector registers today at zero memory traffic; externalizing it is a net loss. The spare cores are worth more running an independent RVV GEMM — and indeed a well-threaded RVV build already beats IME end-to-end on this part, because IME is confined to cluster 0.

Portability. The optimization is specific to the SpaceMiT IME microkernel, but the packing it feeds (a pre-transposed B-is-$N \times K$ tile layout, the same

shape `mmt4d` and ggml-repack produce) is the layout any framework matrix-unit backend expects. The X60 IME microkernel is a building block that MLAS (hence ONNX Runtime), XNNPACK (TensorFlow Lite, PyTorch mobile), and ruy (TensorFlow Lite) do not ship for this hardware; the block-scale accounting here transfers directly, as does the conclusion that scale-build instruction count — not the matrix instruction — is where a serial-pipe matrix core spends the quantization overhead.

Where the win *would* land. A quieter or single-tenant board would surface the ~3–4 % prefill gain; more usefully, a part whose matrix unit spans more than one cluster's cores (lifting the cluster-0 cap) would let IME win end-to-end, at which point a 5 % kernel improvement is 5 % of a path that matters.

9. Conclusion

We closed the largest reducible part of the SpaceMiT X60 IME block-scale tax with a register-efficient scale build: eight `vfmul.vv` and two prebuilt row-scale vectors in place of sixteen masked `vfmul.vf`, four vector moves, and their `vsetvli` churn. The change is bit-identical to the stock kernel and ~5 % faster on both `M4` prefill paths, verified on real X60 silicon. It buys **no measurable end-to-end speedup** on this board, because GEMM is only part of prefill and the application-level measurement noise (±15–20 %) is far larger than the achievable gain. Both results are real, and reporting only the first would be misleading. The kernel improvement is free, correct, and upstreamable; its practical value is gated by the microarchitecture (serial IME/FP pipes, register pressure) and the platform (a four-core, shared-L2 IME cluster), not by the kernel.

Artifact availability

The microbenchmark harness (`ime/bench_i8i4.cpp`, with the `chk`/`zp`/`m1` A/B modes), the pure-`s8s8s32` reference (`ime/`), and the optimization itself (`ime/llama-ime1-scalebuild-opt.patch`) are in this repository. All numbers were produced on an Orange Pi RV2 (SpaceMiT K1/X60) at 1.6 GHz, `performance` governor, single core 0 for the microbenchmarks and cluster 0 (`-t4`) for `llama-bench`.

References

1. llama.cpp / ggml — SpaceMiT IME backend (`ggml-cpu/spacemit`, `GGML_CPU_RISCV64_SPACEMIT`; `ime1_kernels.cpp`).
2. SpaceMiT K1/M1 (Ky X60) technical reference — IME `smt.vmadot`, `xsmtvdot` assembler support (binutils ≥ 2.46).
3. RISC-V "V" Vector Extension, version 1.0.

4. Microsoft MLAS `MlasSQNBitGemm` (the `SQ4Bit…CompInt8` quantized-GEMM scheme the llama.cpp kernel names derive from).
5. `opensolvers/benchmarks` — `ime/README.md` (pure `s8s8s32` ceiling, cache-set aliasing) and `ime-llama` study notes.